

O B EOutcome Based Education System (OBE)
Department of Software Engineering
NED University of Engineering & Technology**Theory**

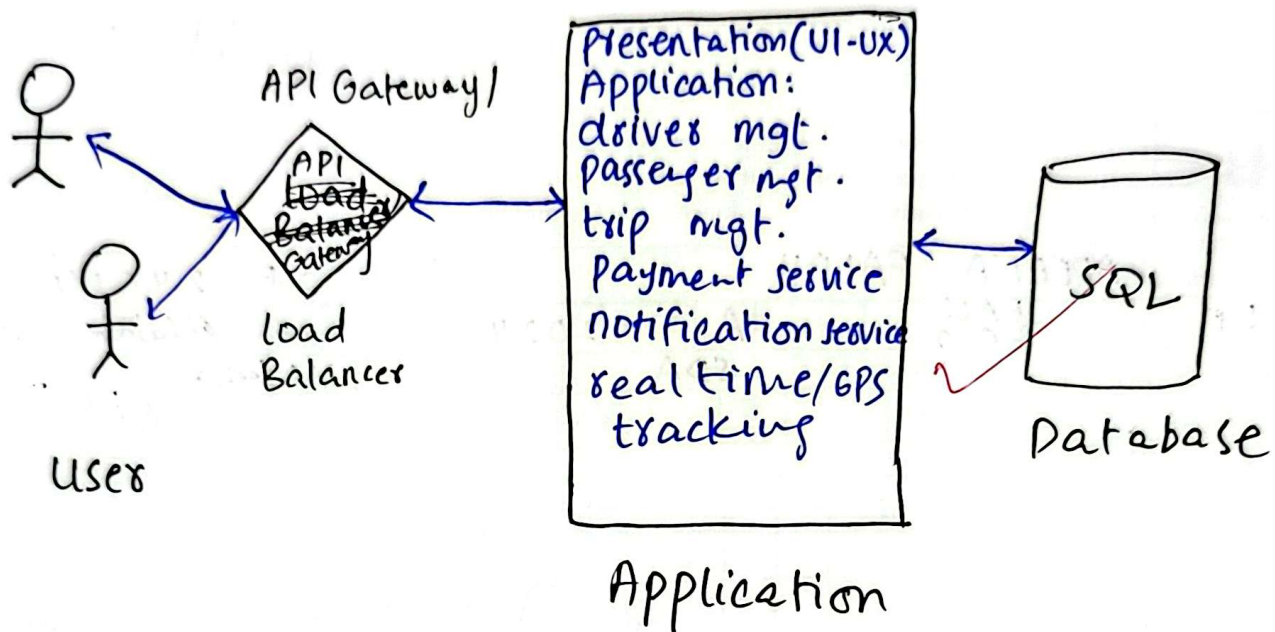
Name (BLOCK Letters)	ADEENA FALAH										Roll Number	SE-240114			
Year	SE	Semester	FALL		2024		Section	A	Batch	2024	Student Sign	afalah			
Course Code							Course Name	SPA			Instructor	MS. Sana			
Test No.							Date					Week			
Mid Term (M)												Room			
DO NOT WRITE IN THIS TABLE (EXAMINER USE ONLY)													DO NOT WRITE IN THIS TABLE (DEO USE ONLY) (OBE MIS SYSTEM)		
CLO	2	2	1	1								Total	Examiner sign with date		
Question	1	2	3	4	5	6	7	8	9	10					
Maximum Marks	6	4	5	5								20			
Achieved Marks	6	4	5	5								10.5			
													Data Entry Operator Name		
													Data Entry Date		
													Data Entry Operator Sign		

Write Your answer below this line

Question no.01

a) Uber's Monolithic Architecture-

Uber is a ride hailing app that first adopted monolithic architecture as a startup. It means all of its services were combined at a single place i.e. a single codebase. All the services for example, driver management, trip management, passenger management, notification service, real time tracking, payment service etc. were connected to a single database i.e. sql. Therefore the UI (presentation layer), Application layer (services) and database had a single deployment unit. The application used API Gateway for communicating to third party services like payment gateway etc.

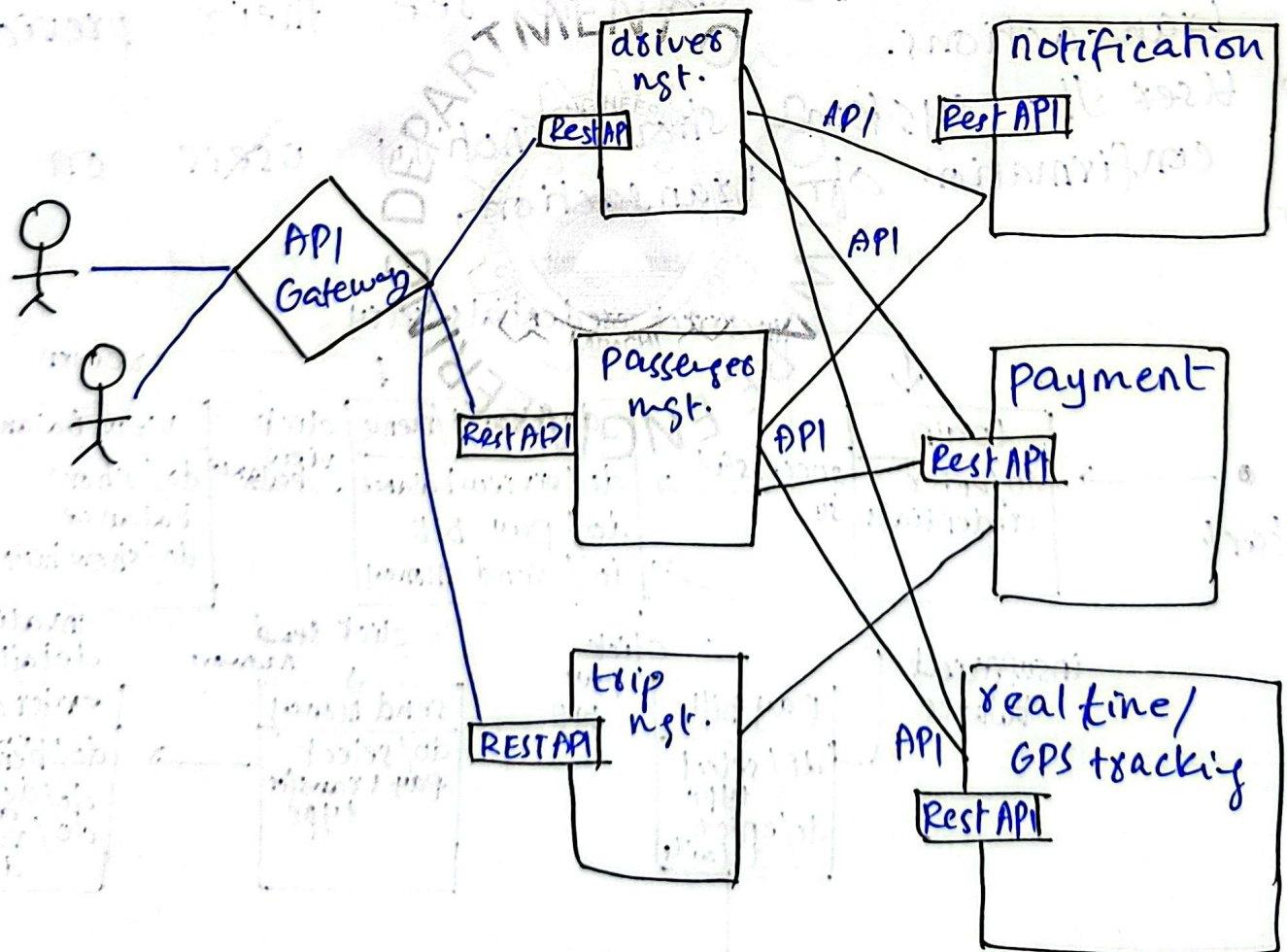


b) Issues:-

- i) ~~Single~~ Database: As the no. of users ~~gradually~~ increased, ~~later~~, one single database was not enough to handle such huge data.
- ii) Scalability: Initially they had ~~had~~ limited services, but when they tried to scale the application by adding more services like (intercity transport etc.) they faced scalability issues.
- iii) Increased overhead: The application was unable to accommodate large no. of users at a single time.
- iv) Debugging & Maintenance/Change Accomodation: It was difficult to debug & fix errors as the whole application needed to be shut ^{down} for even making a small change in any service.

c) Uber's Micro-service Archite^{-chre.}

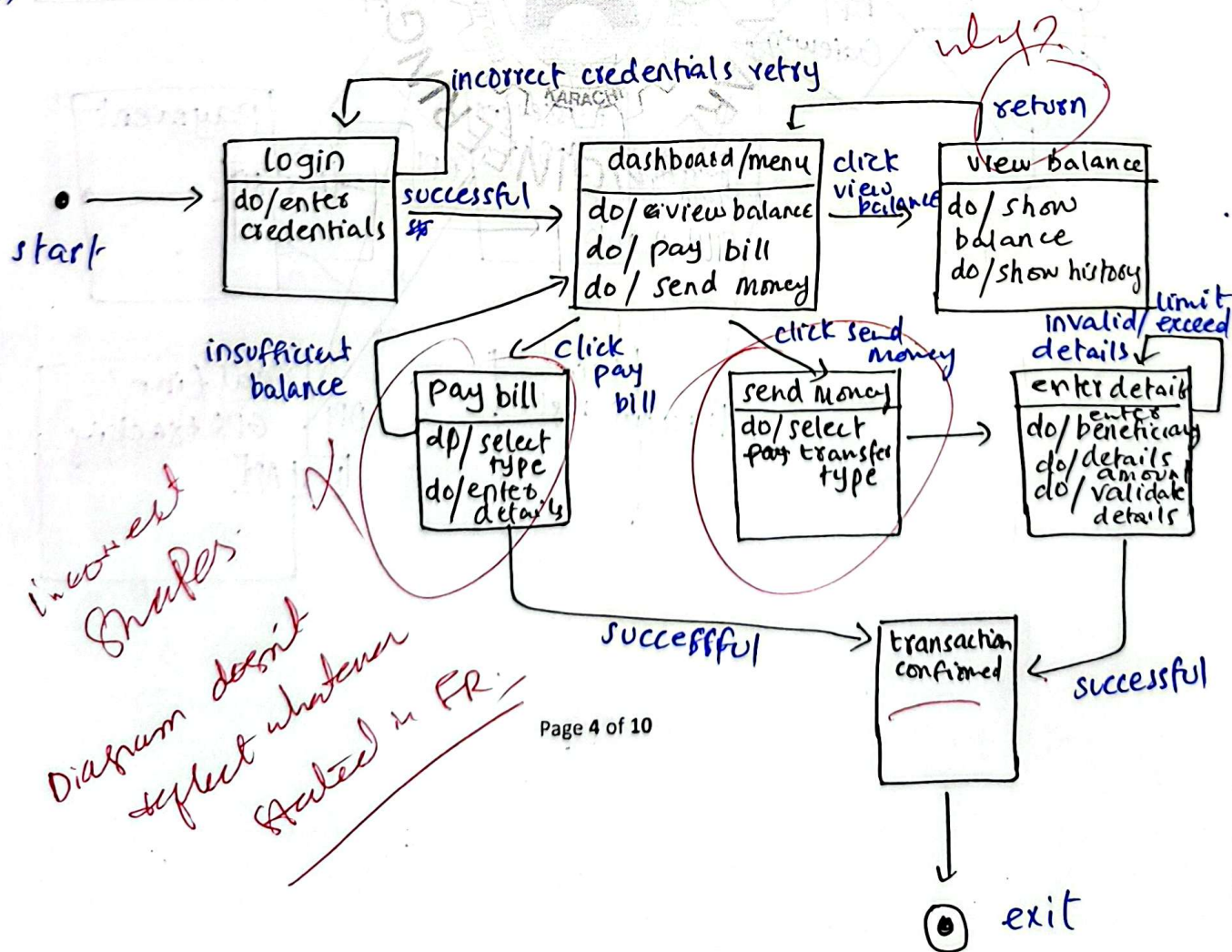
To solve all of these issues, Uber adopted Micro-service architecture in which all the functionalities were divided into separate services each having its own database. The services were able to communicate with each other through APIs. The application was made scalable, easy to maintain & accomodate change while handling a huge am.no. of requests at a time.



Q-3 Easy Paisa Payment Method

-FRS:-

- Users should be able to open account using their CNIC and cell phone number.
- Users should be able to pay different types of bills (such as electricity, gas, water etc)
- Users should be able to send/receive money by entering cell phone / CNIC of beneficiary / recipient.
- Users should be able to see their previous transactions.
- ~~Users~~ The system should notify users on confirmation of transaction.



Q.2) HealthCare Data Management:

The comparasions for given requirements are given below:

This Application will have separate services both for SOA & Microservices Architecture style but SOA will be having single database while

In Micro-services, each service would have its own database. Services like Doctor management, Patient records, rooms/wards allocation, staff task scheduling, payment service, notification service, Appointment service would communicate to each other through well-defined APIs and can accomodate client/user request through single entry point API Gateway along with third party integrations such as payment gateway, electronic health etc.

i) Efficient Search & Retrieval:

To search & fetch data from databases, efficiently Micro-service would be a good architecture due to separate databases for each service. In SOA, it will increase overhead on a single database when multiple services would be initiating request to it.

ii) ~~Scalability~~ Scalability:

Again Microservices win here as growing volume of data and increasing user traffic would be difficult to manage in SOA in single database.

iii) Real-time Collaboration:

Both Architecture have well-defined APIs to coordinate among services so either will work fine. But Microservices ensures reduced latency over SOA to ease collaboration.

iv) Integration with external systems:

To integrate external systems such as payment gateway, health records or diagnostic devices Microservice works better. Although both have API Gateways but Microservices ensures reduced traffic overhead, but both are eligible.

From the above Analysis, I will prefer microservices as it ensures efficiency, scalability, low latency and is able to manage a huge amount of data records.

Q.4

As a senior software architect, I would advice my juniors to:

i) have design knowledge:

they should know designs such as oop, Monolithic, SOA, client server, cloud etc. to excel their job.

ii) Domain knowledge:

They should know the basics of which domain they are working on to make projects and suggest architecture accordingly.

iii) Technical knowledge:

They should have knowledge about IT environments to suggest which technologies to opt such as database, external systems, API integrations/coding.

iv) Methodology knowledge:

Should know different SDLC, waterfall, Agile methodologies for developing software.

Roles of Software Architect

- i) They should align the project & Architecture with business goals of industry. (cost, deadlines etc.)
- ii) They should address all stakeholders to ensure understanding and team work.
- iii) They should deliver a clean, consistent and complete artifact of functional goals upto two level of decomposition.

⇒ Merits of Modularity: (high cohesion, low coupling)

Modularity means to divide different functionalities into separate modules.

i) It increases ^{maintainability for accommodating changes.} ~~cohesion~~. Modules are easy to deploy and develop, (parallel development)

ii) ~~It~~ Scalability: Fixing one module does not interrupt the others functionality. i.e. prevents crashing of whole system.

Demerits:

i) It increases complexity of application. Different services has to take time for communication.

ii) It provides bigger attack surface for security threats if the abstraction is not done properly.